

kodama-cpp: A Portable C++ Library for KODAMA Cross-Validated Accuracy Maximization

Stefano Cacciatore

STEFANO.CACCIATORE@ICGEB.ORG

Bioinformatics Unit, International Centre for Genetic Engineering and Biotechnology

Editor: To be assigned

Abstract

KODAMA searches for latent structure by maximizing the cross-validated predictability of an evolving label vector. We present kodama-cpp, a standalone C++17 library with CPU, CUDA, and Apple Metal backends that moves this accuracy-maximization core out of the R package layer while preserving the methodological contract established by the original KODAMA literature. The library exposes typed KNN and PLS-LDA optimization paths, KNN and PLS-LDA cross-validation kernels, KODAMA matrix construction from either data matrices or supplied KNN graphs, graph construction, embedding helpers, and clustering utilities. Its implementation uses float32 numerical matrices and workspaces, a package-owned CPU HNSW search, package-owned CUDA exact/IVF search and k-means, native Metal exact and IVF search, label-aware SIMPLS PLS-LDA, reusable fold buffers, and independent M-cycle execution. The result is a wrapper-independent numerical backend intended to support reproducible R and Python interfaces, with validation reported separately for the internal cross-validation objective, external label agreement, runtime, and visualization quality. We also document the current validation boundary: the graph-input interface is release-validated as an optional entry point, while performance and quality claims are restricted to measurements reported explicitly in the benchmark tables.

Keywords: KODAMA, unsupervised learning, cross-validation, CUDA, Metal, C++ library, nearest neighbors, PLS-LDA

1 Introduction

High-dimensional data analysis often begins with clustering or dimensionality reduction, but neither operation directly asks whether the discovered groups are predictable from the data. KODAMA takes a different view: a useful labeling of samples should be reproducible by a classifier under cross-validation. The labels are not treated as ground truth; they are optimization variables whose quality is measured by held-out prediction.

The original KODAMA paper introduced this idea as knowledge discovery by accuracy maximization, and the subsequent R package made it available as an unsupervised and semi-supervised feature-extraction tool for noisy high-dimensional data. The Bioinformatics package paper describes the algorithm as repeated random label initialization, iterative maximization of cross-validated accuracy by label swapping, and construction of a dissimilarity matrix from the resulting label vectors. The R documentation exposes the same logic through M independent iterative processes, Tcycle optimization steps, KNN or PLS-DA classifiers, and separate KODAMA.matrix and KODAMA.visualization functions.

kodama-cpp addresses this problem by moving the KODAMA numerical core into a standalone C++17 library with CPU, CUDA, and Apple Metal backends. The library currently focuses on two classifier families for the KODAMA optimization layer, KNN and PLS-LDA, because they provide complementary inductive biases: local neighborhood consistency and low-rank linear discrimination in a latent component space. The implementation is designed so R and Python wrappers can call the same backend without reimplementing the mathematics.

We deliberately frame this contribution as a software-methods paper, not as a claim that KODAMA replaces clustering, semi-supervised learning, or manifold learning. Cross-validated accuracy is the internal optimization criterion used to construct candidate label vectors. External labels, when available, are used only for diagnostics such as adjusted Rand index, silhouette, local purity, and runtime-quality tradeoffs.

Table 1: Main contributions of kodama-cpp.

Contribution	Role in kodama-cpp
Formalization	States KODAMA as cross-validated label predictability, with M independent runs and a derived graph or dissimilarity.
Standalone implementation	Moves the numerical core into a C++17 library so R and Python wrappers share CPU, CUDA, and Metal implementations.
Classifier-specific kernels	Provides clean KNN and PLS-LDA optimization paths instead of wrapper-level branching.
GPU-ready numerics	Uses float32 numerical workspaces, CUDA and Metal nearest-neighbor search, and label-aware SIMPLS PLS-LDA.
Reproducible API	Returns typed predictions, accuracies, labels, graphs, timings, memory, and backend parameters.

2 KODAMA objective

Let X in $\mathbb{R}^{(n \times p)}$ denote the input matrix, c in $\{1, \dots, K\}^n$ a candidate label vector, F a classifier family, and P_i a fold assignment. For each validation sample i , the classifier is trained on samples not in $P_i(i)$ and predicts c_i from x_i . The empirical objective is the held-out accuracy $A(c; X, F, P_i)$. KODAMA searches for a labeling c^* that maximizes this quantity, turning unsupervised structure discovery into a self-guided supervised prediction problem.

One KODAMA run starts from an initial partition. In the historical R interface, this can be one label per sample or a clustering-derived vector W ; a fraction of samples can be selected for each run, with 75% as the documented default. The current C++ interface keeps the same idea but exposes starting labels, splitting, landmarks, and fixed or constrained groups as typed inputs. During each cycle, misclassified samples or constrained groups generate candidate label moves. A move is accepted if it improves the objective, or under a temperature rule that allows exploration early in the run.

The M runs are independent. This independence is important both statistically, because it averages over random initialization and proposal order, and computationally, because backend schedulers can process runs without changing the objective. The final KODAMA

representation is obtained from the ensemble of optimized label vectors: pairs of samples that repeatedly receive compatible labels are pulled closer in the derived graph or dissimilarity, while unstable pairs are weakened.

$$c^* = \arg \max_c A(c; X, F, \Pi), \quad (1)$$

where A is the held-out accuracy of classifier family F under fold assignment Π .

2.1 Label-vector search mechanics

The object optimized by KODAMA is the label vector itself. At any time the algorithm holds a candidate vector y on the landmark samples. A classifier is not used to predict an external truth; instead it is trained to reproduce y under cross-validation. A label vector is therefore good when the labels assigned to held-out samples are predictable from the measured variables. The raw accuracy attached to a candidate, $\text{acc}(y)$, is the fraction of landmark samples for which the held-out prediction $\text{pred}_i(y)$ equals the candidate label y_i .

The first CV pass is run on the initial label vector. This gives both an initial accuracy and a prediction vector. The prediction vector is then used as a proposal guide. If a sample or constrained group repeatedly receives a different predicted label from the classifier, that predicted label is evidence that the current label vector is not aligned with the structure that the classifier can reproduce. KODAMA uses this information to propose a new label vector rather than drawing completely blind random swaps.

Within a cycle, the algorithm chooses a proposal base. In evolutionary mode this is the current accepted vector; otherwise it is the best vector found so far. It samples a number of movable groups according to the adaptive proposal-size schedule. Early cycles can alter many groups, allowing coarse movement through label space. Later cycles alter fewer groups, turning the search into local refinement. For each sampled group, the replacement label is drawn from the empirical distribution of the previous CV predictions among the non-fixed members of that group.

The class-transition step aggregates the same information at label level. It builds a transition matrix N_{ab} counting samples whose current label is a and whose held-out prediction is b . Large off-diagonal counts indicate that the classifier consistently maps one candidate class into another. KODAMA may therefore propose merging unstable source labels into predicted destination labels, while rejecting moves that collapse the solution to a single class or do not reduce unstable fragmentation.

The many-to-one transition proposal uses the same matrix N . For a source class a and a destination b , the expected count under independence is $n_a n_b / n$. A candidate absorption is considered only when N_{ab} exceeds this expectation and also exceeds the number of samples that remain in class a . Candidate destinations are sampled with probability proportional to their positive surplus, and the selected source classes are mapped to the destination only if the move reduces the number of active classes without collapsing all labels into one class.

For the PLS-LDA path, an additional automatic coarsening proposal may be applied before the CV evaluation. It computes the class entropy $H = -\sum_k p_k \log p_k$ and the effective number of classes $K_{\text{eff}} = \exp(H)$. Fragmentation is measured as $\max(0, \log(K / \max(1,$

$K_{\text{eff}}))$). Classes with high transition instability and small movable size are proposed for merging into their strongest predicted destination; the merge budget is $\text{ceil}(T_{\text{frag}} \max(0, K - K_{\text{eff}}))$, where $T_{\text{frag}} = \text{fragmentation} / (\text{fragmentation} + \text{weighted_transition_entropy} + 1)$. This keeps the proposal rule tied to CV confusion rather than to external labels.

After these proposal moves, the proposed vector is evaluated by exactly one new CV pass. The resulting raw accuracy is stored for the best accepted vector, while the acceptance score may include generic guards against degenerate labelings. With guarded diversity, S starts as $A \sqrt{1 - \sum_k p_k^2}$, where A is raw CV accuracy and p_k is the class proportion. With automatic coarsening, S is reduced by $(1 - A) \max(H / \log n, \text{fragmentation} / (1 + \text{fragmentation}))$. These score terms steer the search away from trivial or excessively fragmented vectors without changing the underlying CV classifier.

The best vector is updated whenever the proposal score improves. In evolutionary mode, a separate current vector is also maintained. A worse proposal can become the current vector with probability $\exp((\text{score_new} - \text{score_current}) / \tau.t)$, where $\tau.t$ cools toward zero as cycles progress and is also smaller when current accuracy is already high. This simulated-annealing-style rule permits early exploration but makes late-cycle changes conservative. Repeating this process for M independent runs gives an ensemble of high-accuracy label vectors rather than relying on a single local optimum.

2.2 Raw accuracy, acceptance score, and diagnostics

It is useful to distinguish three quantities that appear in the implementation. The first is the raw held-out CV accuracy $A(y)$, which is the classifier reproducibility score reported for a label vector. The second is the proposal acceptance score $S(y)$, which is the stochastic-search objective actually used to choose moves when degeneracy guards are enabled. The third is an external diagnostic, such as ARI or embedding compactness, which is never optimized by KODAMA but is reported when reference labels are available.

This distinction is necessary because a high raw CV accuracy alone can be achieved by a collapsed or overly coarse label vector. Such a vector may be easy for the classifier to reproduce but poor as a representation of latent structure. The guarded diversity factor and class-coarsening penalties are therefore part of the current standard KODAMA search score in `kodama-cpp`: they change which proposals are accepted, while the reported accuracy remains the raw held-out classifier accuracy.

The same distinction also clarifies how to compare KNN and PLS-LDA. KNN can produce very stable local partitions with high raw accuracy, whereas PLS-LDA can favor labelings that are predictable in a low-dimensional discriminant space. For that reason, the benchmark reports both raw CV accuracy and label-quality diagnostics, rather than treating any single scalar as a complete measure of KODAMA quality.

2.3 Reproducible KODAMA.matrix procedure

The public `KODAMA.matrix` routine is reproducible from the following option definitions and run-level procedure. The description names the C++ options because the R and Python wrappers should expose the same contract.

Table 2: KODAMA.matrix options that determine the optimization.

Option	Role in the algorithm
M / runs	Number of independent KODAMA runs. KODAMAMatrixOptions::runs defaults to 100. Run r uses seed seed + r and writes one length-n label vector to the result matrix.
Tcycle / cycles	Number of proposal/evaluation cycles inside each run. KODAMAMatrixOptions::cycles defaults to 20 for compatibility with the historical interface; the benchmark-quality setting used in this manuscript is 100. CoreOptions accepts the same role when the core optimizer is called directly.
landmarks	Maximum number of samples optimized directly in one run. The default is 10000; if the requested value is at least n, the implementation uses $\text{ceil}(0.75n)$, then caps the value to $[2, n-1]$.
splitting	Initial number of label classes for each run. If not supplied, splitting is 100 for $n \leq 40000$ and 300 otherwise. The run initializes labels by k-means with $\min(\text{splitting}, \text{number of landmarks})$ clusters.
constrain	Optional group vector. Empty means one movable group per sample. Non-empty constraints keep all members of a group in the same CV fold and make the group the unit of label proposal.
fixed	Optional binary vector. Fixed samples are included in CV but are not relabeled by proposal moves.
graph_neighbors	Number of neighbors retained in the final sparse graph. The actual k is $\min(\text{graph_neighbors}, \text{landmarks}, \text{floor}(0.75n - 1))$ after enforcing $k \geq 1$.

2.3.1 RUN-LEVEL OPTIMIZATION RULE

1. For each run $r = 1, \dots, M$, create a random generator with seed seed + r. Copy the input matrix into float32 working storage. Select up to landmarks representative samples by running k-means on all rows with landmarks centers for 10 iterations, then sampling one representative from each non-empty cluster.
2. Initialize the label vector on the selected landmarks. If starting labels are supplied, subset them to the landmarks and apply constrained-majority relabeling when constraints are present. Otherwise run k-means on the landmark matrix with $\text{init_k} = \max(2, \min(\text{splitting}, \text{number of landmarks}))$ for 10 iterations.
3. Build CV folds on the landmark set. With no constrain vector, samples are assigned to folds directly. With a constrain vector, each group is assigned as a whole; stratified folds use the group-majority label and non-stratified folds shuffle groups. KODAMA.matrix currently sets the internal CV folds to non-stratified for both KNN and PLS-LDA core paths.
4. At cycle t, choose the proposal base. In evolutionary-chain mode, the base label vector and prediction vector are the current accepted state; otherwise they are the current best state. These carried predictions drive the label proposal, and the proposed label vector is then evaluated by one CV pass.
5. Sample proposal groups. Let G be the number of groups. With adaptive proposal size enabled, $p.t = (t + 1) / (Tcycle + 1)$, $s.t = p.t^2 (3 - 2 p.t)$, $\theta.t = 1 - s.t$, and

- $q_{\max}(t) = 1 + \text{floor}((G - 1) \theta_{t,t})$. Draw q uniformly from $\{1, \dots, q_{\max}(t)\}$ and sample q groups without replacement. Without adaptive proposal size, q is uniform on $\{1, \dots, G\}$.
6. For each sampled group, collect the non-fixed members. Their candidate replacement labels are the previous CV predictions for those members. The replacement label is drawn from the empirical frequency of those predicted labels, and all eligible members of the group are relabeled together.
 7. Apply class-level transition proposals. The transition matrix has entries $N_{ab} = \#\{i: \text{current label } a, \text{ CV prediction } b\}$. The many-to-one proposal selects targets with positive surplus over the independence expectation $n_a n_b / n$ and relabels one or more unstable source classes into the selected target, while rejecting moves that collapse to one class or fail to reduce the class count. For PLS-LDA, an additional coarsening move may merge small unstable classes according to the same transition matrix.
 8. Evaluate the proposed labels with exactly one CV pass. Let A_t be the fraction of landmark samples whose proposed label equals the held-out CV prediction. The implementation records the raw accuracy attached to the selected best vector, while the default KODAMA.matrix acceptance score guards against degenerate labelings by multiplying A_t by $\sqrt{1 - \sum_k p_k^2}$, where p_k is the current class proportion. When class coarsening is enabled, an additional parsimony term based on label entropy and effective class count is subtracted.
 9. Update the best vector whenever the proposal score improves. In evolutionary-chain mode, also update the current vector if the score improves the current score, or with probability $\exp((\text{score} - \text{current_score}) / \tau_{t,t})$, where $\tau_{t,t} = \max(1e-9, 0.10 \max(0, 1 - \text{current_accuracy}) (1 - (t + 1) / T_{\text{cycle}}))$.
 10. After T_{cycle} cycles, project the optimized landmark labels back to all samples. For the KNN path, each non-landmark receives the majority label among its labeled landmark neighbors in the global graph. For the PLS-LDA path, the final PLS-LDA model predicts the non-landmarks. If constraints are present, the final labels are replaced by the majority label inside each constraint group.

2.3.2 FINAL GRAPH AND DISSIMILARITY

The C++ implementation stores the final KODAMA representation as a sparse neighbor graph rather than materializing a dense n by n matrix by default. First, it constructs a global KNN graph on the original samples using the selected metric and `graph_neighbors`. This unmodified graph is returned as `base.knn`.

Let $\hat{c}_i(r)$ be the final label of sample i from run r . For each retained edge (i,j) with original distance d_{ij} , compute V_{ij} as the number of runs where both endpoint labels are nonzero, and S_{ij} as the number of those valid runs where the two labels agree. If $V_{ij} = 0$ or $S_{ij} = 0$, the corrected edge distance is set to infinity. Otherwise $a_{ij} = S_{ij} / V_{ij}$ and $d'_{ij} = (1 + d_{ij}) / a_{ij}^2$. Each neighbor row is then sorted by d'_{ij} . The resulting graph is the KODAMA-corrected graph used by `KODAMA.visualization`.

A dense dissimilarity can be derived from the same agreement statistic by evaluating the formula for all sample pairs. The library keeps the sparse graph form because downstream UMAP, openTSNE, and graph clustering only need local neighborhoods, and the sparse form is the scalable object shared by the available backends.

3 Implementation architecture

The core library exposes C++ functions rather than wrapper-specific entry points. A `MatrixView` accepts double or float inputs; internally, the analysis matrix is stored in float32 buffers to match GPU kernels and reduce memory traffic. The public API is grouped into cross-validation kernels, core label optimization, KODAMA matrix construction, graph/embedding utilities, and clustering routines.

KNN kernels have strict backend-specific implementations. The dependency-free CPU path uses a package-owned float32 HNSW implementation adapted from the MIT-licensed FAISS organization. CUDA remains opt-in, but its exact search, recall-tuned IVF-Flat search, k-means, and label-aware SIMPLS/LDA are package-owned float32 code requiring only CUDA Toolkit libraries. Apple Metal uses only system frameworks and provides exact and recall-tuned IVF-Flat search. All paths implement the same voting objective. In the KODAMA optimizer, neighbor work is organized so the search structure can be reused where the mathematics permits. This is the main difference from repeatedly launching wrapper-level nearest-neighbor searches.

PLS-LDA uses a SIMPLS strategy following the fastPLS implementation rather than a simplified SVD approximation. CPU, CUDA, and Metal paths operate on float32 matrices and compute class-label cross-products directly instead of constructing dense one-hot response matrices where possible. The requested component count is treated as the evaluated component count whenever mathematically feasible, rather than being replaced by an internal model-selection shortcut. The library exposes no PLS-cKNN classifier; KODAMA optimization is limited to KNN and PLS-LDA.

For graph inputs, where neighbor indices and distances are supplied by the caller, KODAMA keeps the KNN optimizer directly on the supplied graph and avoids rebuilding the neighbor search. If the PLS-LDA optimizer is requested without the original feature matrix, the graph is converted into a PLS-compatible float32 representation with the standard self-tuning normalized Laplacian transform: local row-wise distance scales define edge weights, the graph is symmetrized and degree-normalized, and randomized power iterations produce features for the same PLS-LDA core. This is the only supported graph-to-feature path in the public API.

The graph-only PLS-LDA transform is deterministic conditional on the supplied seed. For row i , σ_i is the median finite neighbor distance, falling back to the row mean and then to $1e-6$ when needed. Each directed edge has weight $w_{ij} = \exp(-d_{ij}^2 / \max(1e-12, \sigma_i \sigma_j))$; reciprocal edges are added by taking the larger weight, and the operator is symmetrically normalized as $D^{-1/2} W D^{-1/2}$. Random Gaussian feature columns are repeatedly multiplied by this sparse operator, orthonormalized for at least eight iterations, and finally standardized before entering the ordinary PLS-LDA core.

The implementation deliberately keeps classifier paths clean. KODAMA optimization is exposed as KNN or PLS-LDA; auxiliary cross-validation kernels exist for testing

and benchmarking, but the KODAMA optimizer is not a collection of benchmark-specific branches. The graph, embedding, and clustering utilities are similarly separated from the label-evolution loop.

kodama-cpp architecture

Standalone C++17 core with dependency-light CPU/Metal and opt-in CUDA backends

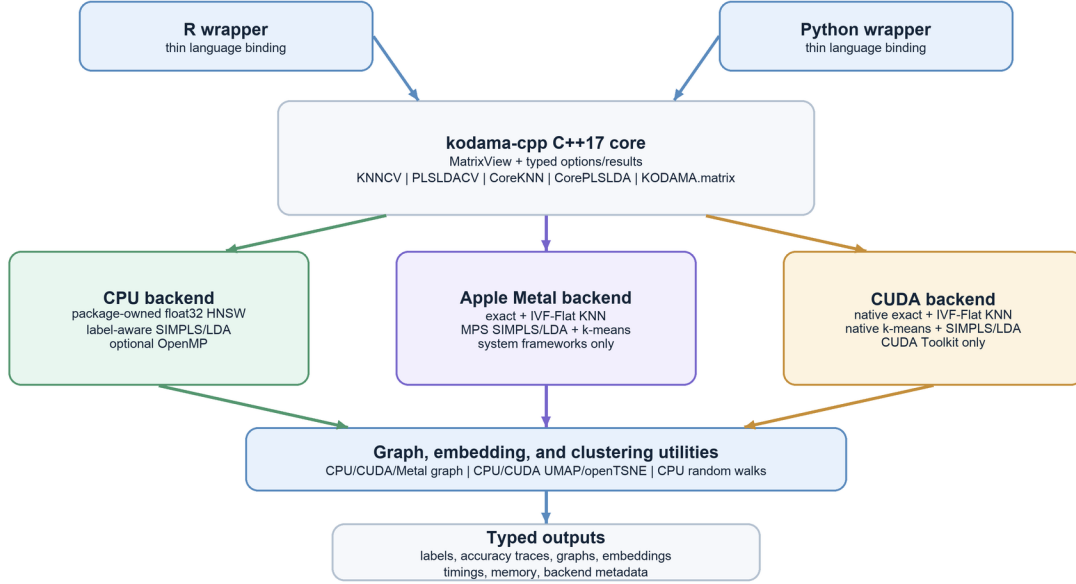


Figure 1: Architecture of kodama-cpp. The C++17 core owns the numerical kernels, CPU, CUDA, and Apple Metal backends, graph utilities, and typed outputs; R and Python remain thin wrappers.

3.1 Computational scaling

3.2 Graph-input KODAMA validation boundary

4 Relationship to public KODAMA literature

The contribution of kodama-cpp is not a new definition of KODAMA. It is a reorganization of the published method into a reusable numerical system. The 2014 PNAS paper is the methodological reference for accuracy maximization. The 2017 Bioinformatics paper is the software reference for the R package, including KNN and PLS-DA classifiers, repeated suboptimal runs, and construction of a dissimilarity matrix from the optimized label vectors. The R documentation is the public reference for defaults such as $M = 100$, $T_{\text{cycle}} = 20$, 75% sample selection, and the output fields dissimilarity, accuracy, proximity, classification vectors, scores, entropy, and landmarks.

The main differences are architectural and computational. The R package owns a complete R workflow and uses R-package dependencies for data handling, nearest neighbors,

Table 3: Public API groups in kodama-cpp.

Layer	Public API	Purpose
Cross-validation	KNNCV, PLSLDACV	Fold assignment, prediction, fold accuracies, confusion matrices, timing, memory, and back-end metadata.
Core optimization	CoreKNN, CorePLSLDA	Independent label-evolution runs that maximize KNN or PLS-LDA cross-validated accuracy.
KODAMA matrix	KODAMA.matrix / KODAMAMatrix	Landmark selection, splitting, M runs, optimized label vectors, KODAMA dissimilarity, and graph output.
Graph-input KODAMA	KODAMA.matrix.graph / KODAMAMatrixFromGraph	Runs the same KNN or PLS-LDA KODAMA optimizer from supplied neighbor indices/distances; KNN reuses the graph directly and PLS-LDA uses self-tuning Laplacian features when the data matrix is unavailable.
Visualization	KODAMA.visualization	UMAP and openTSNE embeddings from standard or KODAMA-corrected neighbor graphs.
Graph and clustering	makeSNNGraph equivalent and random-walk clustering	CPU, CUDA, or Metal neighbor-graph construction followed by an explicit CPU random-walk clustering kernel.

Table 4: Dominant computational work in the KODAMA pipeline.

Component	Dominant work	Implementation implication
KODAMA runs	M independent runs, each with Tcycle proposal cycles plus one initial CV pass.	Embarrassingly parallel over M; each cycle performs exactly one CV evaluation.
KNN core	Neighbor search/build plus $O(M \text{ Tcycle } n_L k)$ voting on landmarks, where n_L is the landmark count and k is the KNN predictor size.	Graph-input KNN removes the search/build term and reuses supplied indices and distances.
PLS-LDA core	$O(M \text{ Tcycle folds } C \cdot \text{PLS}(n_{\text{train}}, p, h))$ for h requested components, plus latent-space LDA scoring.	Float32 label-aware SIMPLS and fold buffers reduce constants but do not change the repeated-CV structure.
Graph correction	$O(M E)$ agreement counting for E retained graph edges, followed by row-wise neighbor sorting.	Sparse by default; dense n by n dissimilarities are avoided for scalability.
Visualization	Runs on the base or KODAMA-corrected sparse graph using UMAP/openTSNE helpers.	Visualization time is reported separately because it is not part of the CV optimization objective.

and visualization. kodama-cpp owns the numerical kernels and exposes them through C++ types so that R and Python wrappers can share one backend. This makes the comparison with the previous version a compatibility question: the objective and output contract should match, while implementation details such as float32 storage, package-owned

Table 5: Interpretation of graph-input KODAMA paths.

Path	Implementation	Interpretation
KNN graph input	Uses supplied neighbor indices/distances directly in CoreKNNGraph and KODAMAMatrixFromGraph.	Accepted as the graph-input path because it preserves the KNN voting mathematics and removes repeated search work.
PLS-LDA graph input with X	When the original data matrix is supplied with the graph, PLS-LDA uses float32 X and the graph supplies initialization/projection support.	Equivalent classifier path to data-input PLS-LDA; graph handling is an input-format extension.
PLS-LDA graph-only input	When X is unavailable, the graph is transformed into self-tuning normalized Laplacian features before CorePLSLDA.	Exposed as an optional fallback, not as evidence that graph-only PLS-LDA matches data-input PLS-LDA in all datasets.
Validation boundary	Wrapper tests exercise both KNN and PLS-LDA graph-input calls; benchmark comparisons must use the same M, Tcycle, seed, and supplied graph.	The manuscript does not use graph-only PLS-LDA results to support data-input KODAMA performance claims.

CPU/CUDA/Metal search, CUDA kernels, and wrapper-independent graph objects are new.

Table 6: Compatibility with the public KODAMA R interface and literature.

Aspect	Public R/literature contract	kodama-cpp
Objective	KODAMA maximizes cross-validated accuracy of an evolving label vector.	Same objective; exposed through CoreKNN and CorePLSLDA.
Independent runs	R documentation reports M iterative processes and Tcycle optimization cycles.	KODAMAMatrixOptions exposes runs and cycles; runs remain independent across CPU, CUDA, and Metal backends.
Sample selection	The documented FUN_SAM default selects 75% of samples per iterative process.	Landmark and splitting controls preserve the same sampling role while exposing it in typed options.
Classifiers	The R package documents KNN and PLS-DA classifiers.	The KODAMA optimizer exposes KNN and SIMPLS + PLS-LDA; component count is explicit.
Output contract	The R interface returns dissimilarity, accuracy, proximity, label vectors, scores, entropy, and landmarks.	The C++ result returns optimized labels, accuracies, graphs, timings, memory, and backend metadata for wrappers.
Architecture	The R package is built around R workflows and R dependencies.	The core is R/Python independent, CMake-installable, and accelerated where available.

4.1 Relationship to semi-supervised learning and cluster validation

KODAMA is related to, but distinct from, semi-supervised learning and weak supervision. Classical semi-supervised graph methods assume that at least some labels are observed and then propagate them under smoothness assumptions, as in Gaussian-field label propagation and local-global consistency. Pseudo-labeling and weak-supervision systems also create or aggregate imperfect labels, but they typically use those labels to train a downstream predictor. KODAMA reverses the emphasis: the label vector itself is the optimization object, and its quality is the ability of a chosen classifier to reproduce it under held-out folds.

The closest evaluation literature is cluster validation by stability or prediction. Methods such as stability-based clustering and prediction strength ask whether a grouping is reproducible under perturbation or held-out prediction. KODAMA uses the same general principle of reproducibility, but internalizes it as the search objective over label vectors. This is why the benchmark reports the internal CV score together with adjusted Rand index, silhouette, local purity, and active class count rather than presenting CV accuracy alone as proof of external correctness.

The manuscript also follows warnings from the model-selection and circular-analysis literature. Cross-validation can be biased when the same data are used both to choose a model and to estimate final performance; feature selection outside the resampling loop is a classic example. In `kodama-cpp`, the internal CV score is intentionally the search criterion, so external claims must come from separate diagnostics, transparent parameter sensitivity, and comparisons with standard UMAP and t-SNE on the same datasets.

Table 7: Literature positioning of KODAMA.

Literature area	Typical goal	Role of KODAMA
Semi-supervised graph learning	Label propagation and local-global consistency start from observed labels and smooth them on a graph.	KODAMA starts from synthetic or supplied candidate labels and optimizes their cross-validated predictability.
Pseudo-labeling and weak supervision	Pseudo-labeling and data programming create imperfect labels to train predictive models.	KODAMA treats the label vector as the object being discovered; the classifier is the measuring instrument.
Cluster stability and prediction strength	Stability methods evaluate whether clusters survive perturbation or prediction on held-out samples.	KODAMA turns held-out predictability into an optimization objective, then reports external diagnostics separately.
Manifold visualization	UMAP and t-SNE optimize low-dimensional layouts from neighbor or affinity structures.	KODAMA supplies a corrected graph to the same visualization step; it does not replace those embedding objectives.
Open-source ML software	JMLR software papers emphasize clear APIs, reproducibility, tests, licensing, and benchmark transparency.	<code>kodama-cpp</code> separates the numerical core from R/Python wrappers and records backend parameters and validation evidence.

5 Evaluation

We evaluate kodama-cpp at three levels. First, kernel benchmarks isolate KNNCV and PLSLDACV from the full KODAMA matrix pipeline so that nearest-neighbor search and PLS-LDA fitting can be interpreted separately. Second, core optimizer benchmarks measure repeated label-vector evolution over multiple seeds. Third, matrix-level validation measures end-to-end KODAMA.matrix calls and reports both cross-validated accuracy and external-label diagnostics where reference labels are available.

The evaluation is intentionally separated into these layers because KODAMA has several scaling regimes. KNN acceleration is dominated by neighbor search and voting. PLS-LDA acceleration is dominated by repeated SIMPLS and LDA work across folds and cycles. The final graph and visualization stages depend primarily on the sparse graph size. Reporting the layers separately makes it possible to attribute a speedup to the part of the implementation that produced it.

The evaluation also separates optimization from validation. The label vector is chosen by internal cross-validation accuracy, but this score is not interpreted as external biological, chemical, or semantic truth. When reference labels are available, they are held outside the optimizer and reported after the fact. This distinction follows the broader cautionary literature on model selection, feature selection bias, and circular analysis.

All timings in this section are wall-clock seconds from the benchmark drivers, and all accuracy values are computed on held-out cross-validation folds. CPU kernel measurements use the configured single-thread CPU path unless explicitly labeled otherwise; CUDA measurements use the same CUDA Toolkit runtime used by the test suite. Tables are intentionally separated by scope: kernel-level rows do not claim end-to-end speedups, and graph-input rows are treated as API validation unless paired with data-input KODAMA results under the same M, Tcycle, seed, and graph settings.

5.1 Benchmark protocol and data coverage

5.2 Evaluation guardrails and ablations

5.3 Dependency-free CPU and Apple Metal validation

The dependency-light backend experiment was run on macOS with CUDA and OpenMP disabled. The CPU build therefore measures the package-owned HNSW and float32 SIMPLS/LDA implementations, while the Metal build links only Apple system frameworks. Exact CPU/Metal equality of the reported classification metrics is the primary acceptance criterion; timings are secondary evidence.

Metal IVF-Flat remains an explicit option rather than an automatic replacement for exact search. On MNIST10k, exact and recall-tuned IVF KNNCV both produced accuracy 0.9490, while elapsed time changed from 2.054 to 1.662 s. A 0.99 pilot-recall target was rejected because accuracy decreased to 0.9482; the accepted automatic pilot target is 0.999. End-to-end KODAMA can still favor exact search when repeated IVF training costs more than the saved query time.

Table 8: Benchmark protocol for the current evaluation.

Item	Value
Date	2026-07-14 UTC for native-CUDA validation; earlier broad benchmark snapshot dated 2026-07-06 UTC
GPU	NVIDIA GeForce RTX 5060 Ti, 16 GB device memory, driver 595.71.05
Build validation	Fresh CUDA 13.2 build succeeded; CTest passed 2/2 configured tests in 1.77 s
Runtime	CUDA Toolkit only for native search/PLS paths; no FAISS, cuVS, RAFT, RMM, or Armadillo link
Data format	RData lists exported to contiguous float32 row-major matrices
CPU setting	Single-thread CPU kernels unless otherwise stated
CV kernels	KNNCV and PLSLDACV measured independently from the full matrix pipeline
Core optimizer	Three seeds per dataset for CoreKNN and CorePLSLDA medians
Reported metrics	Wall time, peak memory where available, CV accuracy, ARI, and active class count

Table 9: Datasets copied into the benchmark data root for the current evaluation.

Dataset	Samples	Variables	Classes
COIL20	1,440	16,384	20
FashionMNIST	70,000	784	10
FlowRepository	5,220,347	32	13
MNIST	70,000	784	10
Macosko2015_retina	44,808	50	12
MetRef	873	375	22
TabulaMuris	70,118	50	56
USPS	11,000	256	10
flow18	1,000,021	11	16
imagenet	1,281,167	1,024	1,000
mass41	965,282	14	17

5.4 CUDA workstation validation

The benchmark run used copied float32 RData datasets spanning $n = 873$ to 5,220,347 samples and $p = 11$ to 16,384 variables. The CUDA machine first rebuilt the project and reran the CTest suite; both configured tests passed. The broad kernel and core tables retain the earlier release-validation snapshot for continuity, whereas the native-CUDA table reports the dependency-distilled search implementation introduced afterward.

The current package-owned CUDA IVF-Flat path was checked on MNIST70k and MetRef with five-fold cosine KNNCV and $k = 10$. On MNIST70k it produced accuracy 0.973857 in 4.233 s with $nlist = 237$ and maximum $nprobe = 32$, compared with 0.973029 in 4.753

Table 10: Evaluation safeguards used to avoid over-interpreting the internal CV objective.

Concern	Manuscript position	Evidence reported or controlled
Circularity	CV accuracy is the internal objective, not an external truth score.	Report ARI, silhouette, local purity, active classes, and standard embedding baselines after optimization.
Overclaiming	Present kodama-cpp as a KODAMA implementation and acceleration library, not a new universal learning paradigm.	State methodological continuity with the 2014 and 2017 KODAMA publications and limit novelty claims to architecture and numerics.
Parameter dependence	M, Tcycle, landmarks, splitting, KNN k, graph k, and PLS components can affect results.	Use predefined sensitivity experiments and report parameter settings in every benchmark table and plot.
Visualization bias	A visually appealing layout is not itself evidence of label quality.	Run classic UMAP/openTSNE and KODAMA-corrected UMAP/openTSNE with the same implementation and record compactness metrics.
Runtime attribution	End-to-end speedups can hide which kernel produced the benefit.	Separate CV kernels, core optimizer, KODAMA.matrix, graph construction, and visualization timings.
Reproducibility	Wrapper behavior must match the standalone core.	Record CMake settings, backend metadata, R CMD check, pytest, CTest, seeds, and benchmark command lines.

s for the recorded former FAISS/cuVS row. On MetRef it produced 0.816724 in 0.195 s, compared with 0.814433 in 0.297 s. These are implementation spot checks rather than repeated-run performance estimates.

The KODAMA.matrix MetRef validation shows why cross-validated accuracy is reported together with label-quality diagnostics. The KNN optimizer reached very high CV accuracy, but the best label vectors collapsed to few classes and had low ARI. The PLS-LDA optimizer had lower CV accuracy but much stronger external-label agreement. We therefore report CV accuracy, number of active classes, ARI or another label-agreement statistic when available, and the downstream embedding compactness as complementary diagnostics.

The dependency-distilled CUDA build links CUDA Toolkit libraries but no FAISS, cuVS, RAFT, or RMM soname. Binary wrappers therefore need to locate the CUDA runtime selected at build time, but do not need to provision the removed search libraries.

5.4.1 EARLIER ACCELERATOR VALIDATION SNAPSHOT

5.4.2 CORE OPTIMIZER MEDIANS

5.4.3 MULTICPU VERSUS GPU KODAMA COMPARISON

The backend comparison is separated by scope. Core-optimizer rows measure the repeated CV label-search kernels without final graph construction, while the KODAMA.matrix row

Table 11: Ablation matrix for the release benchmark.

Ablation	Comparison	Question answered
Classifier	KNN versus PLS-LDA	Tests whether local-neighbor and latent-linear predictability give complementary label vectors.
Graph correction	Standard graph versus KODAMA-corrected graph	Isolates whether the optimized label ensemble improves downstream UMAP/openTSNE structure.
Search depth	Tcycle = 20, 50, 100	Measures whether additional proposal cycles improve CV accuracy or reduce fragmentation.
Ensemble size	M = 20, 50, 100	Measures whether more independent runs stabilize the final graph and best-run diagnostics.
Landmark and splitting controls	Predeclared values, not per-dataset visual tuning	Checks sensitivity of the initial label space and directly optimized sample subset.
Backend	Single-core CPU, 4-core MultiCPU, CUDA, and Apple Metal where available	Separates mathematical equivalence from scheduling and hardware acceleration.
Wrapper parity	C++ core, R wrapper, Python wrapper	Ensures language bindings are thin and do not introduce independent numerical behavior.

Table 12: Dependency-free CPU and native Apple Metal validation. Quality entries are CPU/Metal where two values are shown.

Dataset	Scope	CPU s	Metal s	Speedup	Quality
MetRef	KNNCV	11.145	0.026	425x	accuracy 0.827033 / 0.827033
MetRef	PLSLDACV, 50 components	3.395	1.054	3.2x	accuracy 0.991982 / 0.991982
MetRef	KODAMA KNN, M=10 T=20	21.709	0.171	127x	CV 0.978626; ARI 0.093922; 15 classes
MetRef	KODAMA PLS-LDA, M=10 T=20	178.403	78.746	2.27x	CV 0.983206; ARI 0.835153; 32 classes

includes landmark selection, label search, projection, and graph correction on a small end-to-end validation dataset. This prevents kernel timings from being reported as full pipeline timings.

The rows below therefore state their scope explicitly. CPU rows use the configured CPU backend, CUDA rows use the CUDA backend, and differences in CPU/CUDA accuracy are treated as reproducibility tolerances caused by stochastic search, float32 arithmetic, and approximate nearest-neighbor behavior rather than as evidence of different mathematical objectives.

5.4.4 CLASSIC VERSUS KODAMA VISUALIZATION VALIDATION

Because the final KODAMA object is often interpreted through a two-dimensional layout, we added an explicit visualization comparison against standard UMAP and openTSNE. The driver runs the same embedding implementation on either the ordinary neighbor graph

Table 13: Dependency-distilled native CUDA KNN spot checks.

Dataset	Path	Accuracy	Seconds	nlist	max nprobe	Pilot recall
MNIST70k	native IVF-Flat	0.973857	4.233	237	32	0.994531
MetRef	native IVF-Flat	0.816724	0.195	27	20	0.991406

Table 14: Earlier CPU/CUDA cross-validation kernel measurements retained for release-history continuity. Accuracy is reported as CPU/CUDA.

Dataset	Kernel	CPU s	CUDA s	Speedup	Acc CPU/CUDA	Delta	Comp.
COIL20	KNN CV	5.455	1.435	3.8x	0.917/0.916	-0.000695	-
MNIST	KNN CV	76.769	4.753	16.2x	0.974/0.973	-0.000928	-
MetRef	KNN CV	0.163	0.297	0.6x	0.821/0.814	-0.006873	-
MetRef	PLSLDACV	3.196	0.118	27.2x	0.992/0.992	0.000000	50
USPS	KNN CV	2.273	0.429	5.3x	0.688/0.688	-0.000819	-
USPS	PLSLDACV	3.760	0.158	23.8x	0.684/0.687	+0.002727	50
mass41	KNN CV	366.883	3.692	99.4x	0.912/0.912	-0.000066	-

or the KODAMA-corrected graph, then reports truth-label silhouette, local label purity, KODAMA label ARI, active classes, and wall-clock time. This makes the visualization comparison separate from the CV objective and prevents the evaluation from relying on a single favorable small dataset.

The current CPU visualization validation used KNN KODAMA with $M = 4$ and $T_{\text{cycle}} = 20$ on MetRef, COIL20, and a 3,000-sample USPS subset. It is a harness validation rather than a general claim that KODAMA improves every embedding: KODAMA KNN achieved high CV accuracy on all three datasets, but the KODAMA-corrected layouts did not uniformly improve truth-label silhouette at these short settings. This result supports the manuscript policy of reporting CV accuracy, label agreement, active classes, and classic-vs-KODAMA visualization metrics together.

5.5 Sensitivity of M and T_{cycle}

Because KODAMA has two nested stochastic controls, M and T_{cycle} cannot be justified by visual quality alone. T_{cycle} controls the depth of label-vector evolution inside one run, whereas M controls how many independent local optima enter the final ensemble. The current release-validation experiment uses two benchmark datasets and both KODAMA classifiers: fixing $M = 100$ while varying T_{cycle} in $\{20, 50, 100\}$, and fixing $T_{\text{cycle}} = 100$ while varying M in $\{20, 50, 100\}$. The experiment used landmarks = 100000, splitting = 100, ncomp = 50, and knn.k = 30.

The T_{cycle} sweep supports $T_{\text{cycle}} = 100$ as the final-analysis setting. On the harder benchmark, the PLS-LDA path changed from a fragmented state at $T_{\text{cycle}} = 20$, with median 67 classes and best CV accuracy 0.482, to median 5 classes and best CV accuracy 0.872 at $T_{\text{cycle}} = 100$. The KNN path also improved on the same benchmark, from best CV accuracy 0.773 to 0.863 and from median 23 classes to 9. The easier benchmark showed

Table 15: Medians for the CoreKNN and CorePLSLDA optimization paths. Accuracy and ARI are reported as CPU/CUDA.

Dataset	Core	CPU s	CUDA s	Speedup	Acc	ARI	Classes
MetRef	CoreKNN	0.147	0.122	1.2x	0.963/0.947	0.271/0.313	16/16
MetRef	CorePLSLDA	16.597	0.626	26.5x	0.885/0.838	0.357/0.366	53/48
USPS	CoreKNN	2.282	0.242	9.4x	0.969/0.968	0.216/0.240	59/53
USPS	CorePLSLDA	20.189	1.950	10.4x	0.938/0.910	0.140/0.146	80/77

Table 16: KODAMA core optimizer CPU/CUDA comparison. Scope is reported explicitly because these rows do not include final graph construction.

Dataset	Classifier	Backend	Seconds	CV acc	ARI	Classes	Scope
MetRef	KNN	CPU	0.147	0.963	0.271	16	core optimizer
MetRef	KNN	CUDA	0.122	0.947	0.313	16	core optimizer
MetRef	PLS-LDA	CPU	16.597	0.885	0.357	53	core optimizer
MetRef	PLS-LDA	CUDA	0.626	0.838	0.366	48	core optimizer
USPS	KNN	CPU	2.282	0.969	0.216	59	core optimizer
USPS	KNN	CUDA	0.242	0.968	0.240	53	core optimizer
USPS	PLS-LDA	CPU	20.189	0.938	0.140	80	core optimizer
USPS	PLS-LDA	CUDA	1.950	0.910	0.146	77	core optimizer

smaller best-accuracy changes, but still showed improved median accuracy and reduced fragmentation for PLS-LDA.

The M sweep gives a different interpretation. Increasing M mostly improves the probability of seeing a good independent solution and stabilizes the ensemble; it is not expected to monotonically improve every single-run diagnostic. In these experiments, $M = 50$ was already close to $M = 100$ for several best-accuracy values, but $M = 100$ either tied or improved the best CV accuracy and provided the conservative ensemble size used for final figures. Smaller M remains appropriate for interactive exploration, while $M = 100$ is the reproducible benchmark setting.

6 Availability and reproducibility

kodama-cpp is intended for release under the MIT License in the public tkcaccia/kodama-cpp GitHub repository. The recommended project split is a standalone C++17 core repository, an R wrapper repository, and a Python wrapper repository. This keeps wrapper code thin and makes the numerical library reusable outside either language ecosystem.

Reproducibility is supported through CMake builds, CPU, CUDA, and Metal tests, wrapper validation tests, benchmark scripts, and explicit recording of backend parameters. The C++ core does not depend on R data readers; wrapper packages are responsible for translating host-language objects into contiguous matrices before calling the library.

Table 17: KODAMA.matrix CUDA validation on MetRef with $M = 20$ and $T_{cycle} = 20$.

Classifier	Elapsed s	Runtime s	CV acc	ARI	Med. classes	Class range
KNN	6.849	3.019	0.990840/0.979389	0.123581/0.046694	3	2-28
PLS-LDA	23.795	23.748	0.975573/0.946565	0.795916/0.681493	35	21-42

Table 18: Local CPU preview comparing standard UMAP/openTSNE with KODAMA-KNN corrected graphs. Silhouette and purity are computed against external labels; ARI and class count refer to the best KODAMA label vector.

Dataset	Embedding	Sil. classic	Sil. KODAMA	Purity	ARI	Classes	KODAMA s
MetRef	UMAP	0.051	-0.157	0.533/0.439	0.052	3	0.916
MetRef	openTSNE	0.002	-0.028	0.579/0.499	0.052	3	0.916
COIL20	UMAP	0.494	0.229	0.824/0.764	0.461	20	59.469
COIL20	openTSNE	0.268	0.168	0.803/0.780	0.461	20	59.469
USPS subset	UMAP	0.189	0.011	0.660/0.573	0.239	23	23.741
USPS subset	openTSNE	0.148	0.040	0.662/0.609	0.239	23	23.741

6.1 Installation paths

6.2 Wrapper validation

6.3 License and dependency notes

7 Limitations

The current implementation prioritizes KNN and PLS-LDA because they are central to the KODAMA optimization principle and can be implemented cleanly on CPU, CUDA, and Apple Metal. Additional classifiers are outside this release unless they preserve the cross-validated accuracy objective and can be tested without changing the method for a specific benchmark.

The main engineering limitation is that KODAMA performs many cross-validation fits. This makes memory locality, buffer reuse, label compaction, and GPU scheduling as important as asymptotic complexity. The present release includes float32 buffers, fold reuse, label compaction, CUDA kernels, and native Metal KNN and PLS-LDA kernels, but full accelerator-resident batching of independent M runs remains an engineering extension rather than a claim of the current manuscript.

8 KODAMA pseudocode

Algorithm 1 isolates one independent run inside the M -run ensemble. It starts from one landmark label vector, uses the previous CV predictions to propose label changes, evaluates each proposal with exactly one CV pass, and returns the best label vector found by that run.

Algorithm 1. One independent KODAMA run

Input:
 X_L : landmark matrix for this run

Table 19: Tcycle sensitivity at fixed M = 100. Accuracy entries show Tcycle = 20 to Tcycle = 100.

Dataset	Classifier	Best acc	Median acc	Median classes	Seconds
Benchmark A	KNN	0.900 -i 0.906	0.844 -i 0.897	4 -i 3	23.4 -i 23.7
Benchmark A	PLS-LDA	0.898 -i 0.911	0.683 -i 0.838	9 -i 4	113.7 -i 527.9
Benchmark B	KNN	0.773 -i 0.863	0.701 -i 0.809	23 -i 9	47.0 -i 51.7
Benchmark B	PLS-LDA	0.482 -i 0.872	0.465 -i 0.829	67 -i 5	143.9 -i 596.7

Table 20: M sensitivity at fixed Tcycle = 100. Entries show M = 20 / 50 / 100.

Dataset	Classifier	Best acc	Median acc	Best-run ARI	Seconds
Benchmark A	KNN	0.906 / 0.905 / 0.906	0.897 / 0.897 / 0.897	0.385 / 0.382 / 0.385	5.3 / 12.2 / 23.7
Benchmark A	PLS-LDA	0.907 / 0.910 / 0.911	0.895 / 0.845 / 0.838	0.377 / 0.378 / 0.381	102.1 / 265.9 / 527.9
Benchmark B	KNN	0.832 / 0.863 / 0.863	0.809 / 0.809 / 0.809	0.413 / 0.352 / 0.352	15.7 / 26.2 / 51.7
Benchmark B	PLS-LDA	0.871 / 0.872 / 0.872	0.812 / 0.815 / 0.829	0.268 / 0.319 / 0.319	116.3 / 301.0 / 596.7

y0: initial label vector on landmarks
groups: movable units, either samples or constrained groups
fixed: optional fixed-label mask
folds: CV fold assignment
F: classifier in {KNN, PLS-LDA}
Tcycle: proposal/evaluation cycles

Output:

y_best: optimized labels for this run
pred_best: CV predictions attached to y_best
acc_best: raw CV accuracy of y_best

```

pred0 <- CrossValidate(F, X_L, y0, folds)
acc0 <- Accuracy(y0, pred0)
score0 <- ObjectiveScore(y0, acc0)
y_best <- y0; pred_best <- pred0; acc_best <- acc0; score_best <- score0
y_current <- y0; pred_current <- pred0; acc_current <- acc0; score_current <- score0

```

```

for t = 1,...,Tcycle do
  if evolutionary_search then
    y_base <- y_current; pred_base <- pred_current
  else
    y_base <- y_best; pred_base <- pred_best
  end if

  y_prop <- y_base
  qmax <- ProposalBudget(t, Tcycle, number_of_groups)
  q <- UniformInteger(1, qmax)
  selected_groups <- SampleWithoutReplacement(groups, q)

```

```

for each group g in selected_groups do
  E <- {i in members(g): fixed[i] != 1}
  if E is empty then continue
  replacement <- SampleEmpirical(pred_base[E])
  y_prop[E] <- replacement
end for

```

```

y_prop <- ClassTransitionProposals(y_prop, pred_base, fixed)
pred_prop <- CrossValidate(F, X_L, y_prop, folds) # exactly one CV pass
acc_prop <- Accuracy(y_prop, pred_prop)

```

Table 21: Implementation claims and validation evidence.

Feature claim	Where implemented	Test proving it	Benchmark proving benefit
Float32 numerical paths	MatrixView in include/kodama/kodama.hpp; DenseF PLS-LDA workspaces in src/plscv.cpp; CUDA and Metal device buffers remain float32.	tests/test_cv.cpp and tests/test_metal.cpp check float32 KNCV/PLSLDACV outputs, backend metadata, requested components, and accuracy thresholds.	The benchmark suite loads contiguous float32 matrices and records CPU, CUDA, and Metal runtime and accuracy separately.
Package-owned CPU HNSW	src/native_knn.cpp implements float32 HNSW search used by KNNCV, CoreKNN, graph construction, and KODAMA.matrix without linking FAISS.	Dependency-free CPU CTest covers folds, prediction accuracy, graph construction, and KODAMA optimization; no FAISS build option remains.	On MetRef, native CPU KNCV reproduced accuracy 0.827033; runtime evidence is reported independently from accelerator paths.
CUDA nearest-neighbor search	src/native_cuda_backend.cu implements package-owned float32 exact KNN, signed-hash IVF-Flat, GPU k-means, resident inverted lists, and exact-pilot recall tuning; KNNCV.CUDA and CoreKNN call it directly.	CUDA tests explicitly exercise exact and IVF index types, tuning metadata, constrained folds, float32 CoreKNN, and end-to-end KODAMAMatrix.CUDA initialization. A clean build and ldd audit exclude FAISS/cuVS/RAFT/RMM.	Native IVF spot checks produced 0.973857 in 4.233 s on MNIST70k and 0.816724 in 0.195 s on MetRef, both above the recorded former FAISS/cuVS accuracy rows.
Label-aware SIM-PLS PLS-LDA	src/plscv.cpp uses label/class inputs in CPU, CUDA, and Metal float32 SIMPLS paths; accelerator implementations avoid dense one-hot responses where possible.	CPU/CUDA/Metal tests check PLSLDACV sizes, constrained folds, selected component reporting, backend metadata, and accuracy thresholds.	CUDA PLSLDACV rows report 27.2x speedup on MetRef; Metal reproduced 50-component MetRef accuracy exactly with a 3.2x speedup over the dependency-free CPU build.

Table 22: Implementation claims and validation evidence (continued).

Feature claim	Where implemented	Test proving it	Benchmark proving benefit
Reusable fold and data buffers	PLSFoldXCacheF in src/plscv.cpp caches fold assignments, train/validation indices, scaled matrices, and CUDA Gram buffers; CoreKNN precomputes fold neighbors.	tests/test_cv.cpp checks CoreKNN/CorePLSLDA result sizes, worker/scheduler metadata, and that optimization does not decrease initial CV accuracy.	CorePLSLDA rows report 26.5x speedup on MetRef and 10.4x speedup on USPS for the CUDA core path.
Native Apple Metal backend	src/metal_backend.mm owns persistent Metal state, exact and IVF-Flat KNN, Lloyd k-means, MPS matrix products, label-aware SIMPLS, and latent-space LDA.	tests/test_metal.cpp exercises exact/IVF KNNCV, graphs, CoreKNN/CorePLSLDA, matrix KODAMA, graph-input KODAMA, and explicit rejection of unavailable Metal clustering.	MetRef exact KNN and PLS-LDA matched CPU accuracy; MNIST10k recall-tuned IVF preserved exact accuracy while reducing KNCV time from 2.054 to 1.662 s.
Graph-input KODAMA	The graph-input entry points in src/kodama_matrix.cpp accept supplied neighbor indices/distances; the KNN path reuses the graph and the PLS-LDA graph-only path builds self-tuning Laplacian features.	R and Python wrapper tests exercise matrix_graph with KNN and PLS-LDA, and C++ tests cover graph construction, clustering, and visualization outputs.	Graph-input is presented as an optional API. The manuscript separately states that graph-only PLS-LDA should not be interpreted as equivalent to data-input PLS-LDA without matched benchmark evidence.

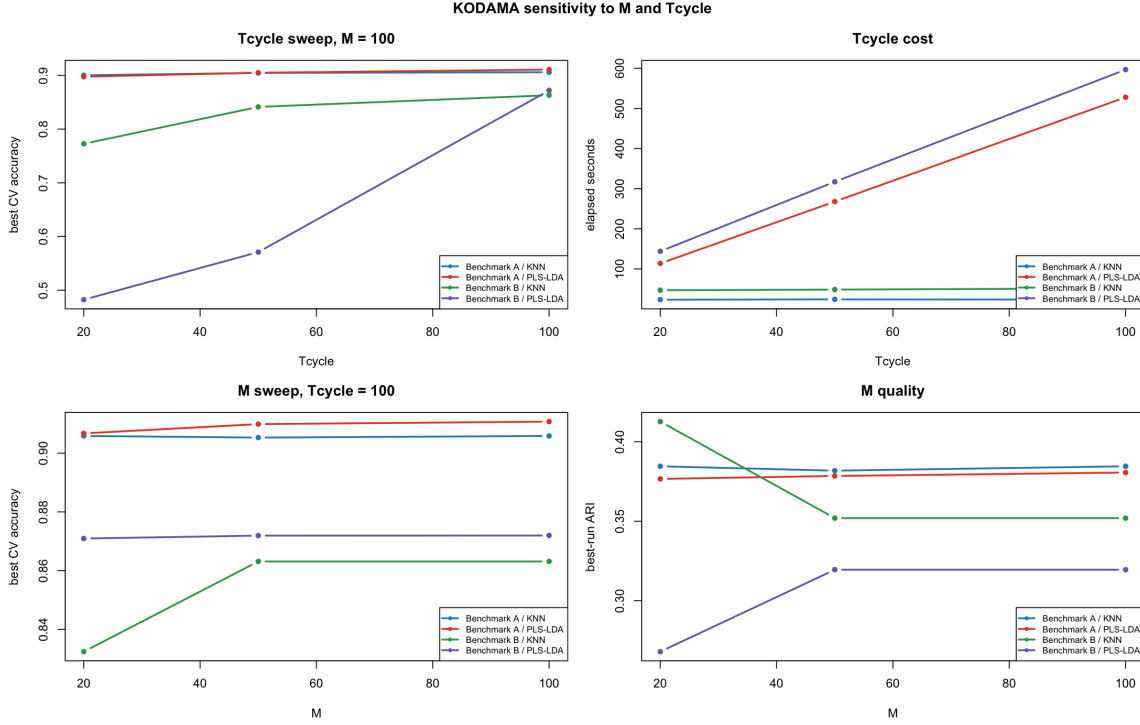


Figure 2: Sensitivity of KODAMA quality and runtime to Tcycle and M. Tcycle = 100 is supported by convergence of the label-vector search; M = 100 is used as a conservative independent-run ensemble size.

Table 23: Release-validation evidence tracked by the project.

Area	Current evidence
Correctness	CTest passed on dependency-free CPU, Apple Metal, and CUDA builds; tests cover constrained folds, predictions, confusion matrices, requested component counts, and backend parity.
Numerics	Float32 MatrixView validation tests exercise CPU, CUDA, and Metal KNN/PLS-LDA paths with typed backend metadata; a fresh CUDA configure/build verifies that SIMPLS has no Armadillo or double-fallback dependency.
Performance	Kernel-level KNNCV/PLSLDACV timings are separated from core optimizer and KODAMA.matrix timings in the benchmark outputs.
Release	CMake install targets, wrapper build scripts, benchmark drivers, and third-party license notices are present; the GitHub commit hash and release tag identify the reviewed software snapshot.

```

score_prop <- ObjectiveScore(y_prop, acc_prop)

if score_prop > score_best then
  y_best <- y_prop; pred_best <- pred_prop
  acc_best <- acc_prop; score_best <- score_prop

```

Table 24: Installation and runtime checks for the standalone core and wrappers.

Target	Command or check
C++ CPU	Configure with CMake using <code>KODAMA_ENABLE_CUDA=OFF</code> and <code>KODAMA_ENABLE_METAL=OFF</code> ; then build, test, and install the exported target.
C++ Metal	On macOS, configure with <code>KODAMA_ENABLE_METAL=ON</code> while CUDA is off; then build and run CTest. Apple system frameworks are linked automatically.
C++ CUDA	Activate an environment containing the CUDA toolkit; configure with <code>KODAMA_ENABLE_CUDA=ON</code> ; then build and run CTest. FAISS, cuVS, RAFT, RMM, Armadillo, and external graph-clustering libraries are not required.
R wrapper	Set <code>KODAMA_CPP_ROOT</code> and <code>KODAMA_CPP_BUILD_DIR</code> , then run R CMD INSTALL kodama-r. The wrapper links the installed or selected core build.
Python wrapper	Pass <code>KODAMA_CPP_ROOT</code> and <code>KODAMA_CPP_BUILD_DIR</code> as CMake config settings to pip when installing kodama-python.
Runtime check	Run ctest for the core library; in R call <code>KODAMA.diagnostics()</code> ; in Python import kodama and run the package validation tests.

Table 25: Local CPU/Metal and CUDA validation results for the core and wrapper packages.

Target	Validation result
C++ core, local CPU	Dependency-free build passed 2/2 configured tests on macOS.
C++ core, Apple Metal	Native Metal build passed 3/3 configured tests; a clean external CMake consumer linked the installed package and selected Metal.
C++ core, CUDA	A fresh CUDA 13.2 build without FAISS, cuVS, RAFT, RMM, or Armadillo succeeded on chiamaka and CTest passed 2/2. The linked-soname audit found none of the removed dependencies, and the CMake cache contained no corresponding package option, target, header, or library entry; only the CUDA Toolkit environment directory retained a legacy faissgpu-cuvs label.
R wrapper, local CPU	R CMD build followed by R CMD check <code>-as-cran -no-manual -no-vignettes</code> passed on the source tarball with only the expected new-submission NOTE under LC_ALL=C.
Python wrapper, local CPU	Temporary virtual-environment install against the local CPU build passed pytest: 4/4 tests.

```

end if

if evolutionary_search and
  Accept(score_prop, score_current, acc_current, t, Tcycle, rng) then
  y_current <- y_prop; pred_current <- pred_prop
  acc_current <- acc_prop; score_current <- score_prop
end if

if acc_prop == 1 then break

```

Table 26: Release notes for core components and external dependencies.

Component	Release note
kodama-cpp	MIT license intended for the standalone core release.
KODAMA, fastPLS, and fastEmbedR-derived code	Authored by the same group and relicensed under MIT in the standalone core.
Native CPU HNSW	Package-owned source adapted from the MIT-licensed FAISS organization; no FAISS link dependency.
Apple Metal	Uses only Foundation, Metal, and MetalPerformanceShaders system frameworks; FAISS and Faiss-mlx source acknowledgements are retained.
Native CUDA	Package-owned exact/IVF KNN, k-means, and SIMPLS/LDA use CUDA Toolkit libraries without FAISS, cuVS, RAFT, RMM, Armadillo, or external graph-clustering links.
R and Python wrappers	Thin language bindings in separate repositories, released under licenses compatible with the core.

end for

return y_best, pred_best, acc_best

8.1 KODAMA.matrix ensemble

Algorithm 2 repeats Algorithm 1 independently M times, projects each optimized run back to all samples, and builds the final KODAMA-corrected graph from agreement across the optimized label vectors.

Algorithm 2. KODAMA.matrix ensemble

Input:

X: n by p data matrix
F: classifier in {KNN, PLS-LDA}
M: number of independent runs
Tcycle: proposal/evaluation cycles per run
landmarks, splitting, graph_neighbors
optional starting_labels, constrain, fixed

Output:

C: M by n matrix of optimized label vectors
A: length-M vector of best raw CV accuracies
G_K: KODAMA-corrected neighbor graph

```

X32 <- float32(X)
G0 <- KNNGraph(X32, graph_neighbors)
for r = 1,...,M do
  rng <- RNG(seed + r)
  L <- SelectLandmarks(X32, landmarks, rng)
  groups <- ConstrainedGroups(constrain[L])
    or one singleton group per landmark
  y0 <- InitialLabels(X32[L], starting_labels[L], splitting, groups, rng)
  folds <- AssignCVFolds(groups, y0)

  y_best, pred_best, acc_best <- OneIndependentRun(
    X32[L], y0, groups, fixed[L], folds, F, Tcycle, rng)

  C[r, ] <- ProjectLabelsToAllSamples(y_best, X32, L, F, groups, G0)

```

```

    A[r] <- acc_best
  end for

  G_K <- ReweightGraphByLabelAgreement(G0, C)
  return C, A, G_K

ObjectiveScore(y, a):
  score <- a
  if guarded_diversity then
    p_l <- class proportions in y
    score <- score * sqrt(1 - sum_l p_l^2)
  end if
  if class_coarsening then
    score <- score - (1 - a) * Parsimony(y)
  end if
  return score

Accept(s_new, s_old, a_old, t, Tcycle, rng):
  if s_new >= s_old then return TRUE
  tau <- max(1e-9, 0.10 * max(0, 1 - a_old) * (1 - t / Tcycle))
  return Uniform(0, 1, rng) < exp((s_new - s_old) / tau)

```

8.2 KODAMA.matrix.graph from supplied neighbors

Algorithm 3 keeps the same label-evolution objective while replacing the initial neighbor search with caller-supplied indices and distances. KNN uses the supplied graph directly; PLS-LDA uses self-tuning Laplacian features only when the original feature matrix is not supplied.

Algorithm 3. KODAMA.matrix.graph from supplied neighbors

```

Input:
  I, D: n by k neighbor indices and distances
  optional X: original n by p data matrix
  F: classifier in {KNN, PLS-LDA}
  M, Tcycle, landmarks, splitting
  optional starting_labels, constrain, fixed

Output:
  C: M by n matrix of optimized label vectors
  A: length-M vector of best raw CV accuracies
  G_K: KODAMA-corrected graph on the supplied neighborhoods

GO <- NormalizeExternalGraph(I, D)
if X is supplied then
  X_work <- float32(X)
else if F == PLS-LDA then
  X_work <- SelfTuningLaplacianFeatures(G0, ncomp)
else
  X_work <- empty
end if

for r = 1,...,M do
  rng <- RNG(seed + r)
  L <- SelectLandmarksFromGraphOrData(G0, X_work, landmarks, rng)
  groups <- ConstrainedGroups(constrain[L])
    or one singleton group per landmark
  y0 <- InitialLabelsFromGraphOrData(G0[L], X_work[L], starting_labels[L],
    splitting, groups, rng)
  folds <- AssignCVFolds(groups, y0)

  if F == KNN then

```



```

    y_best, pred_best, acc_best <- Algorithm1(G0[L], y0, groups, fixed[L], folds, KNN, Tcycle)
  else
    y_best, pred_best, acc_best <- Algorithm1(X_work[L], y0, groups, fixed[L], folds, PLS-LDA, Tcycle)
  end if

  C[r, ] <- ProjectLabelsToAllSamples(y_best, X_work, L, F, groups, G0)
  A[r] <- acc_best
end for

G_K <- ReweightGraphByLabelAgreement(G0, C)
return C, A, G_K

```

Acknowledgments and Disclosure of Funding

The authors thank contributors to the KODAMA, fastPLS, and fastEmbedR software projects.

References

- Christophe Ambroise and Geoffrey J. McLachlan. Selection bias in gene extraction on the basis of microarray gene-expression data. *Proceedings of the National Academy of Sciences*, 99(10):6562–6566, 2002.
- Asa Ben-Hur, Andre Elisseeff, and Isabelle Guyon. A stability based method for discovering structure in clustered data. In *Pacific Symposium on Biocomputing*, pages 6–17, 2002.
- Stefano Cacciatore and Leonardo Tenori. *KODAMA: Knowledge Discovery by Accuracy Maximization*, 2026. R package documentation, CRAN.
- Stefano Cacciatore, Claudio Luchinat, and Leonardo Tenori. Knowledge discovery by accuracy maximization. *Proceedings of the National Academy of Sciences*, 111(14):5117–5122, 2014.
- Stefano Cacciatore, Leonardo Tenori, Claudio Luchinat, Phillip R. Bennett, and David A. MacIntyre. Kodama: an r package for knowledge discovery and data mining. *Bioinformatics*, 33(4):621–623, 2017. doi: 10.1093/bioinformatics/btw705.
- Olivier Chapelle, Bernhard Schölkopf, and Alexander Zien. *Semi-Supervised Learning*. MIT Press, 2006.
- Sijmen de Jong. Simpls: an alternative approach to partial least squares regression. *Chemometrics and Intelligent Laboratory Systems*, 18(3):251–263, 1993.
- Lawrence Hubert and Phipps Arabie. Comparing partitions. *Journal of Classification*, 2: 193–218, 1985.
- Jeff Johnson, Matthijs Douze, and Herve Jegou. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 2019.

- Ron Kohavi. A study of cross-validation and bootstrap for accuracy estimation and model selection. In *Proceedings of the Fourteenth International Joint Conference on Artificial Intelligence*, pages 1137–1143, 1995.
- Nikolaus Kriegeskorte, W. Kyle Simmons, Patrick S. F. Bellgowan, and Chris I. Baker. Circular analysis in systems neuroscience: the dangers of double dipping. *Nature Neuroscience*, 12:535–540, 2009.
- Dong-Hyun Lee. Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks. In *ICML Workshop on Challenges in Representation Learning*, 2013.
- George C. Linderman, Manas Rachh, Jeremy G. Hoskins, Stefan Steinerberger, and Yuval Kluger. Fast interpolation-based t-sne for improved visualization of single-cell rna-seq data. *Nature Methods*, 16:243–245, 2019.
- Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020.
- Leland McInnes, John Healy, and James Melville. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv:1802.03426*, 2018.
- Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d’Alche Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research. *Journal of Machine Learning Research*, 22(164):1–20, 2021.
- Alexander Ratner, Christopher De Sa, Sen Wu, Daniel Selsam, and Christopher Ré. Data programming: Creating large training sets, quickly. In *Advances in Neural Information Processing Systems*, 2016.
- Peter J. Rousseeuw. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, 1987.
- Sören Sonnenburg, Mikio L. Braun, Cheng Soon Ong, Samy Bengio, Leon Bottou, Geoffrey Holmes, Yann LeCun, Klaus-Robert Müller, Fernando Pereira, Carl Edward Rasmussen, Gunnar Rätsch, Bernhard Schölkopf, Alexander Smola, Pascal Vincent, Jason Weston, and Robert C. Williamson. The need for open source software in machine learning. *Journal of Machine Learning Research*, 8:2443–2466, 2007.
- Robert Tibshirani and Guenther Walther. Cluster validation by prediction strength. *Journal of Computational and Graphical Statistics*, 14(3):511–528, 2005.
- Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9:2579–2605, 2008.
- Sudhir Varma and Richard Simon. Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics*, 7:91, 2006.

Dengyong Zhou, Olivier Bousquet, Thomas Navin Lal, Jason Weston, and Bernhard Schölkopf. Learning with local and global consistency. In *Advances in Neural Information Processing Systems*, 2003.

Xiaojin Zhu, Zoubin Ghahramani, and John Lafferty. Semi-supervised learning using gaussian fields and harmonic functions. In *Proceedings of the Twentieth International Conference on Machine Learning*, pages 912–919, 2003.